Assignment 9: Machine Learning with Google colab

Overview

This assignment introduces students to fundamental Machine Learning and Artificial Intelligence concepts using Google Colab and open-source Python libraries. Students will perform hands-on activities in Computer Vision, Deep Learning, Natural Language Processing (NLP), Speech Processing, and Document OCR.

Learning Area	AWS Service	Google Colab Tool
Computer Vision	Rekognition	YOLOv8 + EasyOCR
Deep Learning	SageMaker	PyTorch
NLP	Comprehend	Transformers + spaCy
Speech Processing	Transcribe + Polly	Whisper + gTTS
Document Analysis	Textract	EasyOCR
Cloud Storage	S3	Google Drive
ML Development	SageMaker Notebook	Google Colab

Deliverables

Each student must submit the following:

1. **Presentation Slides (10–35 slides maximum)**
    - Title slide with student name(s), course, and assignment title.
    - Clear explanation of each completed task.
2. **Presentation Guidelines**
    - Use professional formatting (legible fonts, consistent colors, organized layout).

- Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals.
- Provide screenshots or videos as evidence of work completed.
- Keep presentations within the time limit (6–9 minutes).
- Be prepared to answer questions about the process and challenges.

3. Task Completion (60 points total)

Each task is evaluated based on correctness, completeness, and evidence (e.g., screenshots and description of output).

4. Presentation Quality (30 points total)

- Clear structure (results, discussion, and question answer)
- Slides visually organized and professional, for example, adding pictures, transitions, animations
- Screenshots included as evidence

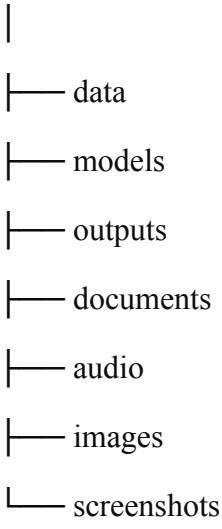
5. Professionalism & Delivery (10 points total)

- Confident and clear presentation delivery
- Ability to answer questions from given questions
- **Submission Instructions**
  - Submit slides (PowerPoint or PDF) to Moodle by the deadline.

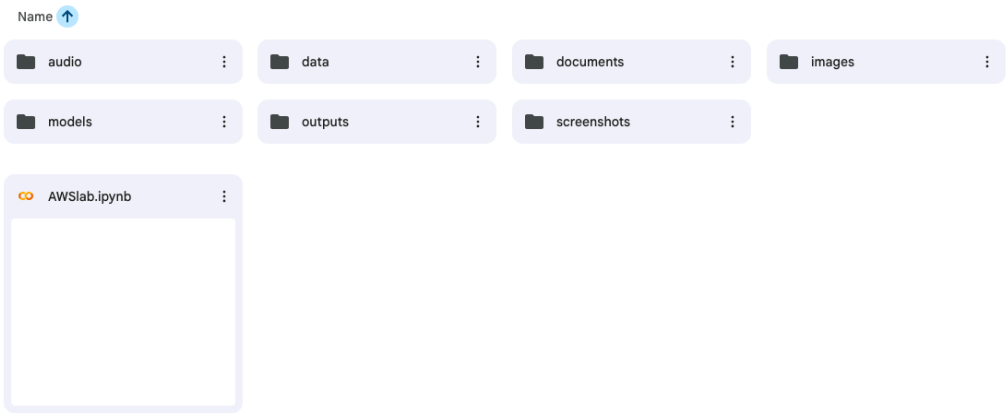
Task 1: Setup google collab

1. Open your Google Drive and create one folder in that name as **machine_learning_lab**.
   a. Now under that create a different subfolder. The architecture of your **machine_learning_lab** looks like as:

machine_learning_lab



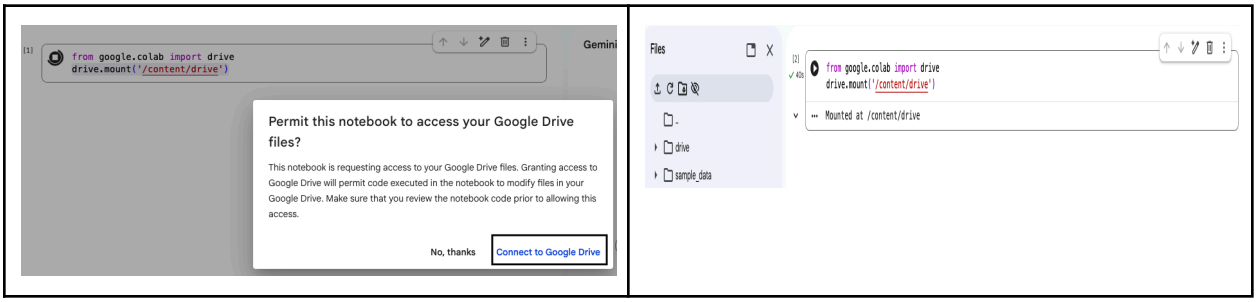
2. Open Google colab and create a new notebook and name it as **machine_learning_lab (**remember the notebook file should be created under machine_learning_lab folder**).**



3. Now Mount Google Drive by using the following code in the notebook. paste and Run it

```
from google.colab import drive
drive.mount('/content/drive')
```

After mounting the drive, the output will be: `Mounted at /content/drive`.



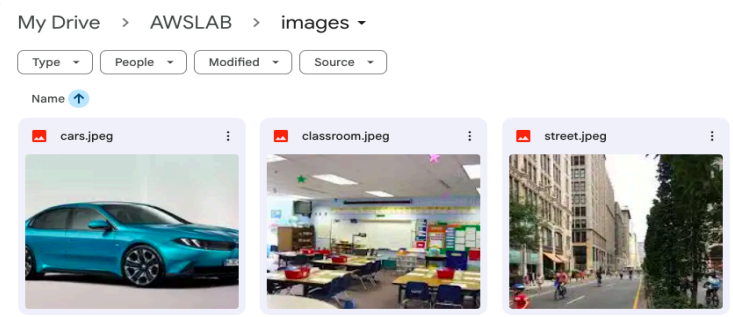
4. Define the path. Copy and paste the following code in your collab notebbok

```
BASE_DIR = "/content/drive/MyDrive/machine_learning_lab"
DATA_DIR = f"{BASE_DIR}/data"
MODEL_DIR = f"{BASE_DIR}/models"
OUTPUT_DIR = f"{BASE_DIR}/outputs"
DOC_DIR = f"{BASE_DIR}/documents"
AUDIO_DIR = f"{BASE_DIR}/audio"
IMAGE_DIR = f"{BASE_DIR}/images"
```

Task 2A: Apply computer vision

1. Object Detection

a. Download some images and store them in the images directory of the drive. Such as car.jpg, classroom.jpg, street.jpg



b. Install Yolo using the following command into the google collab notebook.

```
!pip install ultralytics --quiet
```

c. import yolo library using the code: `from ultralytics import YOLO`

d. Load the model and save it under the models directory.

```
model = YOLO("yolov8n.pt")
import shutil
shutil.copy(
    "yolov8n.pt",
    f"{MODEL_DIR}/yolov8n.pt"
)
print("Model saved successfully")
```

e. Deduct the object by recognising each image using Yolo.

```
from ultralytics import YOLO
import os
# Load YOLO model
load_model = f"{MODEL_DIR}/yolov8n.pt"
# Supported image formats
extensions = (".jpg", ".jpeg", ".png")
# Get all images from folder
image_files = [
    f for f in os.listdir(IMAGE_DIR)
    if f.lower().endswith(extensions)
]
for image_file in image_files:
    image_path = os.path.join(
        IMAGE_DIR,
        image_file
    )
    print(f"\n{'='*50}")
    print(f"Image: {image_file}")
    print(f"{'='*50}")
```

```
results = model(image_path)
for box in results[0].boxes:
    cls = int(box.cls)
    conf = float(box.conf)
    print(
        f"Label: {model.names[cls]:15s} "
        f"Confidence: {conf:.2%}"
    )
```

f. Display the results in the table format such as:

Label	Confidence
car	98%
chair	95%
person	91%

2. Text detection (OCR)

a. Install EasyOCR using the command: `!pip install easyocr --quiet`

b. Download any signboard images and upload the images under the Images folder of the drive.

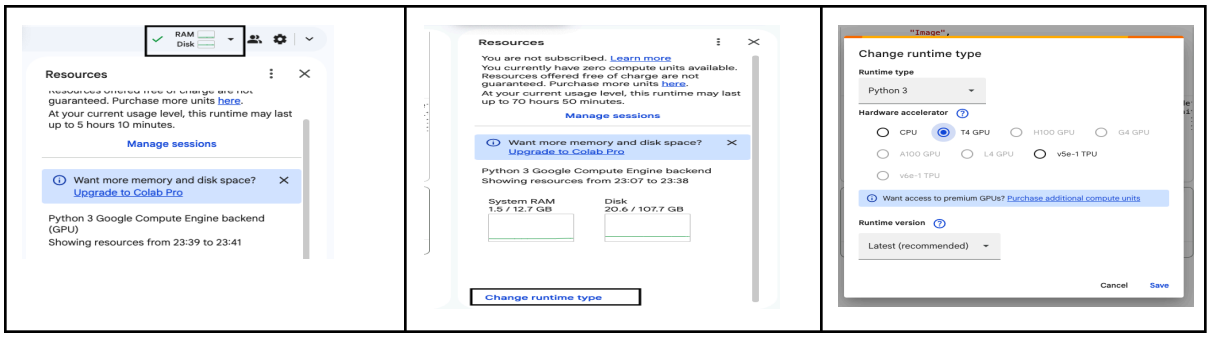
c. Initialize Reader , Run OCR, recognize the text, display the text and save the results in the output folder.

```
import os
import pandas as pd
import easyocr
reader = easyocr.Reader(['en'])
records = []
extensions = (".jpg", ".jpeg", ".png")
image_files = [
    f for f in os.listdir(IMAGE_DIR)
    if f.lower().endswith(extensions)
]
for image_file in image_files:
    image_path = os.path.join(
        IMAGE_DIR,
```

```
            image_file
        )
        result = reader.readtext(image_path)
        for item in result:
            detected_text = item[1]
            confidence = round(item[2] * 100, 2)
            records.append([
                image_file,
                detected_text,
                confidence
            ])
df = pd.DataFrame(
    records,
    columns=[
        "Image",
        "Detected Text",
        "Confidence (%)"
    ]
)
print(df)
df.to_csv(
    f"{OUTPUT_DIR}/ocr_results.csv",
    index=False
)
print("OCR results saved successfully")
```

TASK 2B: AlexNet on MNIST

1. First enable the GPU from the existing collab notebook by following the steps. If GPU is unavailable then use CPU.



2. Verify the device.

```
import torch
device = torch.device(
"cuda" if torch.cuda.is_available()
else "cpu"
)
print(device)
```

3. Import the libraries and load the created directory

```
import torch
import torchvision
from torchvision import datasets
from torchvision import transforms
BASE_DIR = "/content/drive/MyDrive/machine_learning_lab"
DATA_DIR = f"{BASE_DIR}/data"
MODEL_DIR = f"{BASE_DIR}/models"
OUTPUT_DIR = f"{BASE_DIR}/outputs"
DOC_DIR = f"{BASE_DIR}/documents"
AUDIO_DIR = f"{BASE_DIR}/audio"
IMAGE_DIR = f"{BASE_DIR}/images"
```

4. Prepare the dataset:

```
transform = transforms.Compose([
transforms.Resize((224,224)),
transforms.Grayscale(num_output_channels=3),
transforms.ToTensor()
])
```

5. Download MNIST dataset

```
train_dataset = datasets.MNIST(
    root=DATA_DIR,
```

```
    train=True,
    download=True,
    transform=transform
)
test_dataset = datasets.MNIST(
    root=DATA_DIR,
    train=False,
    download=True,
    transform=transform
)
```

6. Create Small Dataset

```
from torch.utils.data import Subset
train_dataset = Subset(
    train_dataset,
    range(1000)
)
test_dataset = Subset(
    test_dataset,
    range(200)
)
```

7. Create DataLoaders

```
from torch.utils.data import DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)
test_loader = DataLoader(
    test_dataset,
    batch_size=32
)
```

8. Load Alexnet

```
from torchvision import models
model = models.alexnet()
model.classifier[6] = torch.nn.Linear(
    4096,
    10
```

```
)
model = model.to(device)
```

9. Use Loss and optimizer to optimize the model

```
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

10. Train the model

```
epochs = 1
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(
            outputs,
            labels
        )
        loss.backward()
        optimizer.step()
print("Training Complete")
```

11. Evaluate the model

```
correct = 0
total = 0
model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predicted = outputs.max(1)
        total += labels.size(0)
```

```
            correct += (
                predicted == labels
            ).sum().item()
    accuracy = 100 * correct / total
    print(
        "Accuracy:",
        accuracy
    )
```

12. Save the model.

```
torch.save(
model.state_dict(),
f"{MODEL_DIR}/alexnet_mnist.pth"
)
```

Task 3: Perform NLP operations with transformers

1. Install the necessary libraries by using existing colab notebook:

```
!pip install transformers
!pip install spacy
!python -m spacy download en_core_web_sm
```

2. Use this sample text for experiment

```
text = """
John purchased a laptop from Amazon.
The laptop performance is excellent.
Battery life is average.
"""
```

3. Perform the sentiment analysis

```
from transformers import pipeline
sentiment = pipeline(
    "sentiment-analysis"
)
result = sentiment(text)
print(result)
```

4. Identify the entity in the given text

```
import spacy
```

```
nlp = spacy.load(
"en_core_web_sm"
)
doc = nlp(text)
for ent in doc.ents:
print(
ent.text,
ent.label_
)
```

TASK 4: Speech Processing

1. Download the audio file from the given link and upload into the audio folder of the google drive.
   https://www.kaggle.com/datasets/pavanelisetty/sample-audio-files-for-speech-recognition
2. Install the necessary library `!pip install openai-whisper`
3. Load the model.
```
import whisper
model = whisper.load_model(
"tiny"
)
```
4. Transcribe the audio file using whisper.
```
result = model.transcribe(
   f"{AUDIO_DIR}/harvard.wav"
)
print(result["text"])
```
5. Save the output of the recognised text from the audio file.
```
with open(
   f"{DOC_DIR}/transcript.txt",
   "w"
) as f:

   f.write(
       result["text"]
   )
```
6. Install the gtts. and convert into the speech of the recognised text.
```
!pip install gtts


from gtts import gTTS
tts = gTTS(
    result["text"],
    lang="vi"
)
```

```
tts.save(
    f"{AUDIO_DIR}/output.mp3"
)
```

Task 5: Document OCR

1. Download any bill receipt image file and upload into the document folder of the google drive.
2. Install easyocr if now installed in the colab notebook
```
!pip install easyocr --quiet
```
3. Copy and paste the given code to recognise the text given in to the bill.
```
import easyocr
reader = easyocr.Reader(['en'])
result = reader.readtext(
f"{DOC_DIR}/receipt.jpg"
)
```

Task 6 – Reflection Questions

1. How did the trained AlexNet model perform on the MNIST dataset, and what factors may have influenced the model's accuracy?
2. Which task (Computer Vision, Deep Learning, NLP, Speech Processing, or Document OCR) did you find most interesting, and how can it be applied in a real-world scenario?